HW3

TING-YU CHANG

Problem 1

a.

```
library(foreign)
```

```
Warning: package 'foreign' was built under R version 4.4.3
```

```
# read the data
aux <- read.xport("AUX_I.xpt")
dem <- read.xport("DEMO_I.xpt")

# merge the data
merged <- merge(aux, dem, by = "SEQN", all = FALSE)

names(merged) <- tolower(names(merged))
dim(merged)
```

```
[1] 4582  119
```

b.

1. gender

```
table(merged$riagendr, useNA = 'ifany')
```

```
   1    2
2176 2406
```

```r
# rename
names(merged)[names(merged) == "riagendr"] <- "female"

# convert to 0 = male and 1 = female
merged$female <- merged$female - 1

# create a table
table(merged$female, useNA = 'ifany')
```

```
   0    1
2176 2406
```

gender : no missing value, rename, no need to make binary variables factors.

2. citizenship status

```r
table(merged$dmdcitzn, useNA = 'ifany')
```

```
   1    2    7    9 <NA>
3684  884    8    5    1
```

```r
# convert invalid or missing to NA
merged$dmdcitzn[merged$dmdcitzn > 3] <- NA

# convert to a factor
merged$dmdcitzn <- as.factor(merged$dmdcitzn)
levels(merged$dmdcitzn) <- c(
  "Citizen",
  "Not a citizen"
)

# rename
names(merged)[names(merged) == "dmdcitzn"] <- "citizenship"

table(merged$citizenship, useNA = "ifany")
```

```
      Citizen Not a citizen          <NA>
         3684           884            14
```

3.

of children

```
# rename
names(merged)[names(merged) == "dmdhhsza"] <- "numchild"

table(merged$numchild, useNA = "ifany")
```

```
   0    1    2    3
3463  737  317   65
```

4. annual household income

```
table(merged$indhhin2, useNA = 'ifany')
```

```
   1    2    3    4    5    6    7    8    9   10   12   13   14   15   77   99
 101  146  233  262  272  461  439  381  295  244  145   53  458  795   86   63
<NA>
 148
```

```
# convert refused, dk, and overlapping income categories to NA
merged$indhhin2[
  merged$indhhin2 > 15 |
  merged$indhhin2 %in% c(12, 13)
] <- NA

# rename
names(merged)[names(merged) == "indhhin2"] <- "hhinc"

merged$hhinc[merged$hhinc == 14] <- 11
merged$hhinc[merged$hhinc == 15] <- 12
table(merged$hhinc, useNA = 'ifany')
```

```
   1    2    3    4    5    6    7    8    9   10   11   12 <NA>
 101  146  233  262  272  461  439  381  295  244  458  795  495
```

```r
# check the cleaned variable
table(merged$hhinc, useNA = "ifany")
```

```
   1    2    3    4    5    6    7    8    9   10   11   12 <NA>
 101  146  233  262  272  461  439  381  295  244  458  795  495
```

c.

```r
# 1R - Right ear: gender
mod1r <- glm(
  auxtwidr ~ female,
  data = merged,
  family = poisson
)

# 2R - Right ear: gender, citizenship status, number of children, annual household income
mod2r <- glm(
  auxtwidr ~ female + citizenship + numchild + hhinc,
  data = merged,
  family = poisson
)

# 1L - Left ear: gender
mod1l <- glm(
  auxtwidl ~ female,
  data = merged,
  family = poisson
)

# 2L - Left ear: gender, citizenship status, number of children, annual household income
mod2l <- glm(
  auxtwidl ~ female + citizenship + numchild + hhinc,
  data = merged,
  family = poisson
)

# combine coefficients
coef_table <- cbind(
  c(coef(mod1r), rep(NA, length(coef(mod2r)) - length(coef(mod1r)))),
```

```
  coef(mod2r),
  c(coef(mod1l), rep(NA, length(coef(mod2l)) - length(coef(mod1l)))),
  coef(mod2l)
)

# convert to IRR
coef_table <- exp(coef_table)

rownames(coef_table) <- names(coef(mod2l))
colnames(coef_table) <- c(
  "Model 1R",
  "Model 2R",
  "Model 1L",
  "Model 2L"
)

coef_table <- round(coef_table, 3)
mode(coef_table) <- "character"
coef_table[is.na(coef_table)] <- ""

# display a formatted table
knitr::kable(
  coef_table,
  align = "cccc",
  caption = "Incidence Risk Ratios for Poisson Regression Models"
)
```

Table 1: Incidence Risk Ratios for Poisson Regression Models

	Model 1R	Model 2R	Model 1L	Model 2L
(Intercept)	84.317	88.437	84.758	88.821
female	1.009	1.015	1.013	1.019
citizenshipNot a citizen		1.047		1.022
numchild		0.994		0.983
hhinc		0.992		0.993

```
library(pscl)
```

```
Classes and Methods for R originally developed in the
Political Science Computational Laboratory
```

```
Department of Political Science
Stanford University (2002-2015),
by and under the direction of Simon Jackman.
hurdle and zeroinfl functions by Achim Zeileis.
```

```r
# calculate model summary stat
calc_stat <- function(model) {
  c(
    "Sample size" = length(model$fitted.values),
    "Pseudo R^2" = unname(pR2(model)["McFadden"]),
    "AIC" = AIC(model)
  )
}

# combine
fit_table <- cbind(
  calc_stat(mod1r),
  calc_stat(mod2r),
  calc_stat(mod1l),
  calc_stat(mod2l)
)
```

```
fitting null model for pseudo-r2
fitting null model for pseudo-r2
fitting null model for pseudo-r2
fitting null model for pseudo-r2
```

```r
# add col name
colnames(fit_table) <- c(
  "Model 1R",
  "Model 2R",
  "Model 1L",
  "Model 2L"
)

fit_table <- round(fit_table, 3)

knitr::kable(
  fit_table,
  align = "cccc",
  caption = "Model Fit Statistics"
)
```

Table 2: Model Fit Statistics

	Model 1R	Model 2R	Model 1L	Model 2L
Sample size	4149.00	3705.000	4103.00	3665.000
Pseudo R^2	0.00	0.004	0.00	0.003
AIC	96618.48	84916.043	98685.15	86597.959

d.

```
# calculate IRR for females relative to males
exp(mod2l$coef["female"])
```

```
  female
1.018746
```

```
# p-value
summary(mod2l)$coef["female", "Pr(>|z|)"]
```

```
[1] 2.473393e-07
```

Strong evidence of an effect on the IRR scale. The incidence risk ratio differs between males and females. Females have a slightly higher expected Tympanometric width than males, after adjusting for citizenship, number of young children, and household income.

```
library(emmeans)
```

```
Warning: package 'emmeans' was built under R version 4.4.3
```

```
Welcome to emmeans.
Caution: You lose important information if you filter this package's results.
See '? untidy'
```

```
# estimate the predicted Tympanometric width for males and females
gender_emm <- emmeans(
  mod2l,
  ~ female,
  type = "response"
```

```
)

gender_emm
```

```
 female rate    SE  df asymp.LCL asymp.UCL
      0 84.8 0.252 Inf      84.3      85.3
      1 86.3 0.249 Inf      85.9      86.8

Results are averaged over the levels of: citizenship
Confidence level used: 0.95
Intervals are back-transformed from the log scale
```

```
# test the diff
pairs(gender_emm)
```

```
 contrast          ratio      SE  df null z.ratio p.value
 female0 / female1 0.982 0.00353 Inf    1  -5.160 <0.0001

Results are averaged over the levels of: citizenship
Tests are performed on the log scale
```

The comparison has a p-value less than 0.0001, indicating strong evidence that the predicted left-ear Tympanometric width differs between males and females. Females have a higher predicted value than males. Both analyses provide strong evidence of a gender difference.

Problem 2

```
library(DBI)
library(RSQLite)
```

```
Warning: package 'RSQLite' was built under R version 4.4.3
```

```
library(microbenchmark)

sakila <- dbConnect(
  RSQLite::SQLite(),
  "sakila_master.db"
)
```

```r
# check the table
dbListTables(sakila)
```

```
 [1] "actor"                "address"              "category"
 [4] "city"                 "country"              "customer"
 [7] "customer_list"        "film"                 "film_actor"
[10] "film_category"        "film_list"            "film_text"
[13] "inventory"            "language"             "payment"
[16] "rental"               "sales_by_film_category" "sales_by_store"
[19] "staff"                "staff_list"           "store"
```

a. For each store, how many customers does that store have, and what percentage of customers of that store are active in the system?

R

```r
# extract customer table
customer <- dbGetQuery(
  sakila,
  "SELECT * FROM customer"
)

# the # of customers in each store
total_customers <- tapply(
  customer$active,
  customer$store_id,
  length
)

# the percentage of active customers in each store
active_percent <- tapply(
  as.numeric(customer$active),
  customer$store_id,
  mean
) * 100

customer_R <- data.frame(
  store_id = as.numeric(names(total_customers)),
  total_customers = as.numeric(total_customers),
  active_percent = as.numeric(active_percent)
```

```
)

customer_R
```

```
  store_id total_customers active_percent
1        1             326       97.54601
2        2             273       97.43590
```

SQL

```
customer_SQL <- dbGetQuery(
  sakila,
  "
  SELECT store_id,
         COUNT(*) AS total_customers,
         100.0 * AVG(active) AS active_percent
  FROM customer
  GROUP BY store_id
  "
)

customer_SQL
```

```
  store_id total_customers active_percent
1        1             326       97.54601
2        2             273       97.43590
```

compare timing

```
microbenchmark(
  R = {
    customer <- dbGetQuery(
      sakila,
      "SELECT * FROM customer"
    )

    total_customers <- tapply(
      customer$active,
      customer$store_id,
      length
```

```
    )

    active_percent <- tapply(
      as.numeric(customer$active),
      customer$store_id,
      mean
    ) * 100

    data.frame(
      store_id = as.numeric(names(total_customers)),
      total_customers = as.numeric(total_customers),
      active_percent = as.numeric(active_percent)
    )
  },

  SQL = {
    dbGetQuery(
      sakila,
      "
      SELECT store_id,
             COUNT(*) AS total_customers,
             100.0 * AVG(active) AS active_percent
      FROM customer
      GROUP BY store_id
      "
    )
  }
)
```

```
Warning in microbenchmark(R = {: less accurate nanosecond times to avoid
potential integer overflows
```

```
Unit: microseconds
 expr     min       lq     mean   median       uq      max neval
    R 864.895 923.4635 948.3468 943.3895 968.4610 1233.280   100
  SQL 165.558 180.6050 191.7287 189.2560 198.4605  248.624   100
```

The SQL approach was faster than the R approach. Based on the median execution times, the SQL query was approximately five times faster.

b. Generate a table identifying the names and country of each staff member.

R

```
# extract required table
staff <- dbGetQuery(sakila, "SELECT * FROM staff")
address <- dbGetQuery(sakila, "SELECT * FROM address")
city <- dbGetQuery(sakila, "SELECT * FROM city")
country <- dbGetQuery(sakila, "SELECT * FROM country")

# merge staff with address
staff_address <- merge(
  staff,
  address,
  by = "address_id"
)

# merge with city
staff_city <- merge(
  staff_address,
  city,
  by = "city_id"
)

# merge with country
staff_country <- merge(
  staff_city,
  country,
  by = "country_id"
)
```

```
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
```

```
# only staff names and country
staff_R <- staff_country[
  ,
  c("first_name", "last_name", "country")
]

staff_R
```

```
  first_name last_name   country
1        Jon  Stephens Australia
2       Mike   Hillyer    Canada
```

SQL

```
staff_SQL <- dbGetQuery(
  sakila,
  "
  SELECT s.first_name,
         s.last_name,
         co.country
  FROM staff AS s
  JOIN address AS a
    ON s.address_id = a.address_id
  JOIN city AS ci
    ON a.city_id = ci.city_id
  JOIN country AS co
    ON ci.country_id = co.country_id
  "
)

staff_SQL
```

```
  first_name last_name   country
1       Mike   Hillyer    Canada
2        Jon  Stephens Australia
```

compare timing

```
microbenchmark(
  R = {
    staff <- dbGetQuery(sakila, "SELECT * FROM staff")
    address <- dbGetQuery(sakila, "SELECT * FROM address")
    city <- dbGetQuery(sakila, "SELECT * FROM city")
    country <- dbGetQuery(sakila, "SELECT * FROM country")

    staff_address <- merge(
      staff, address,
      by = "address_id"
    )
```

```
    staff_city <- merge(
      staff_address, city,
      by = "city_id"
    )

    staff_country <- merge(
      staff_city, country,
      by = "country_id"
    )

    staff_country[
      ,
      c("first_name", "last_name", "country")
    ]
  },

  SQL = {
    dbGetQuery(
      sakila,
      "
      SELECT s.first_name,
             s.last_name,
             co.country
      FROM staff AS s
      JOIN address AS a
        ON s.address_id = a.address_id
      JOIN city AS ci
        ON a.city_id = ci.city_id
      JOIN country AS co
        ON ci.country_id = co.country_id
      "
    )
  }
)
```

```
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
```

```
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
```

```
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
```

```
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
```

```
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
```

```
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
Warning in merge.data.frame(staff_city, country, by = "country_id"): column
names 'last_update.x', 'last_update.y' are duplicated in the result
```

```
Unit: microseconds
 expr      min       lq      mean   median       uq      max neval
    R 2064.965 2143.542 2249.5892 2172.508 2232.635 3875.320   100
  SQL  107.871  120.704  135.1807  132.348  142.270  282.941   100
```

The single SQL query was much faster than the R approach. Based on the median execution times, the SQL approach was approximately 16 times faster.

c. Identify the name(s) of the film(s) which was/were rented for the highest dollar value. (Assume all costs are in USD regardless of country.)

R

```r
# extract the required tables
payment <- dbGetQuery(sakila, "SELECT * FROM payment")
rental <- dbGetQuery(sakila, "SELECT * FROM rental")
inventory <- dbGetQuery(sakila, "SELECT * FROM inventory")
film <- dbGetQuery(sakila, "SELECT * FROM film")
```

```r
# find the rentals with the highest payment amount
max_payment <- max(payment$amount)

rental_ids <- payment$rental_id[
  payment$amount == max_payment
]

# find the corresponding inventory IDs
inventory_ids <- rental$inventory_id[
  rental$rental_id %in% rental_ids
]

# find the corresponding film IDs
film_ids <- inventory$film_id[
  inventory$inventory_id %in% inventory_ids
]

# find the film titles
film_R <- unique(
  film$title[film$film_id %in% film_ids]
)

film_R
```

```
[1] "FLINTSTONES HAPPINESS" "MIDSUMMER GROUNDHOG"   "MINE TITANS"
[4] "SCORPION APOLLO"       "SHOW LORD"             "STING PERSONAL"
[7] "TIES HUNGER"           "TRAP GUYS"             "VIRTUAL SPOILERS"
```

SQL

```r
film_SQL <- dbGetQuery(
  sakila,
  "
  SELECT DISTINCT f.title
  FROM payment AS p
  JOIN rental AS r
    ON p.rental_id = r.rental_id
  JOIN inventory AS i
    ON r.inventory_id = i.inventory_id
  JOIN film AS f
    ON i.film_id = f.film_id
```

```
  WHERE p.amount = (
    SELECT MAX(amount)
    FROM payment
  )
  ORDER BY f.title
  "
)

film_SQL
```

```
                  title
1 FLINTSTONES HAPPINESS
2   MIDSUMMER GROUNDHOG
3            MINE TITANS
4        SCORPION APOLLO
5              SHOW LORD
6         STING PERSONAL
7            TIES HUNGER
8              TRAP GUYS
9       VIRTUAL SPOILERS
```

compare timing

```
microbenchmark(
  R = {
    payment <- dbGetQuery(sakila, "SELECT * FROM payment")
    rental <- dbGetQuery(sakila, "SELECT * FROM rental")
    inventory <- dbGetQuery(sakila, "SELECT * FROM inventory")
    film <- dbGetQuery(sakila, "SELECT * FROM film")

    max_payment <- max(payment$amount)

    rental_ids <- payment$rental_id[
      payment$amount == max_payment
    ]

    inventory_ids <- rental$inventory_id[
      rental$rental_id %in% rental_ids
    ]

    film_ids <- inventory$film_id[
```

```
      inventory$inventory_id %in% inventory_ids
    ]

    unique(
      film$title[film$film_id %in% film_ids]
    )
  },

  SQL = {
    dbGetQuery(
      sakila,
      "
      SELECT DISTINCT f.title
      FROM payment AS p
      JOIN rental AS r
        ON p.rental_id = r.rental_id
      JOIN inventory AS i
        ON r.inventory_id = i.inventory_id
      JOIN film AS f
        ON i.film_id = f.film_id
      WHERE p.amount = (
        SELECT MAX(amount)
        FROM payment
      )
      ORDER BY f.title
      "
    )
  }
)
```

```
Unit: milliseconds
 expr       min        lq      mean    median        uq      max neval
    R 29.070107 29.561943 30.211671 30.06981 30.290000 34.57247   100
  SQL  1.032339  1.057329  1.328278  1.07707  1.607241  2.33618   100
```

Problem 3

```
dat <- read.csv("au-500.csv")
```

a. What percentage of the websites are .com's (as opposed to .net, .com.au, etc)?

```
com_per <- mean(grepl("\\.com$", dat$web)) * 100

com_per
```

```
[1] 0
```

No websites end in .com.

b. What is the most common domain name amongst the email addresses?

```
# extract domain name
domains <- gsub(".*@", "", dat$email)

# the most common domain
domain_counts <- sort(table(domains), decreasing = TRUE)
domain_counts[1]
```

```
hotmail.com
        114
```

The most common email domain is hotmail.com, which appears 114 times.

c. What proportion of company names contain a non-alphabetic character, excluding commas and whitespace.

```
prop_nonalpha <- mean(
  grepl("[^a-zA-Z, ]", dat$company_name)
)

prop_nonalpha
```

```
[1] 0.09
```

9% of company names contain a non-alphabetic character after excluding commas and whitespace.

```
# exclude ampersands from non-alphabetic characters
prop_nonalpha_no_amp <- mean(
  grepl("[^a-zA-Z,& ]", dat$company_name)
)

prop_nonalpha_no_amp
```

```
[1] 0.008
```

The proportion decreases from 9% to 0.8% when ampersands are also excluded.

d.

```
head(dat[, c("phone1", "phone2")], 10)
```

```
          phone1       phone2
1  03-8174-9123 0458-665-290
2  07-9997-3366 0497-622-620
3  08-5558-9019 0427-885-282
4  02-6044-4682 0443-795-912
5  02-1455-6085 0453-666-885
6  08-7868-1355 0451-966-921
7  08-6522-8931 0427-991-688
8  02-5226-9402 0415-961-606
9  07-3184-9989 0411-732-965
10 08-6890-4661 0461-862-457
```

```
# remove dashes
phone1_dig <- paste0(
  substr(dat$phone1, 1, 2),
  substr(dat$phone1, 4, 7),
  substr(dat$phone1, 9, 12)
)

dat$phone1 <- paste0(
  substr(phone1_dig, 1, 4),
```

```
  "-",
  substr(phone1_dig, 5, 7),
  "-",
  substr(phone1_dig, 8, 10)
)

# display the first 10 phone numbers
head(dat[, c("phone1", "phone2")], 10)
```

```
         phone1       phone2
1  0381-749-123 0458-665-290
2  0799-973-366 0497-622-620
3  0855-589-019 0427-885-282
4  0260-444-682 0443-795-912
5  0214-556-085 0453-666-885
6  0878-681-355 0451-966-921
7  0865-228-931 0427-991-688
8  0252-269-402 0415-961-606
9  0731-849-989 0411-732-965
10 0868-904-661 0461-862-457
```

e. Produce a histogram of the log of the apartment numbers for all addresses.

```
# address that end in a number
apt <- dat$address[
  grepl("[0-9]+$", dat$address)
]

# extract the number at the end of each address
apt_nums <- regmatches(
  apt,
  regexpr("[0-9]+$", apt)
)

# convert apt nums to numeric
apartment_numbers <- as.numeric(apt_nums)

hist(
  log(apartment_numbers),
  main = "Log of Apartment Numbers",
```

```
  xlab = "log(Apartment Number)"
)
```



f. Examine whether the apartment numbers appear to follow Benford's law. Do you think the apartment numbers would pass as real data?

```
# extract the leading digit
leading_digit <- substr(
  apartment_numbers,
  1,
  1
)

# count the leading digits
table(leading_digit)
```

```
leading_digit
 1  2  3  4  5  6  7  8  9
12 19 16 13 14 18  8 11 19
```

The leading digits of the apartment numbers appear approximately uniformly distributed rather than decreasing as predicted by Benford's law. Therefore, the apartment numbers do not appear to follow Benford's law and would not be convincing as real-world numerical data.